

75-Day Development Plan

Blockchain Settlement Infrastructure on the Stellar Network
Backend Core · Android (Kotlin) · iOS (Swift) · Web Platform

Client	PathPulse.ai
Service Provider	EngxLab, Mumbai, Maharashtra
Engagement	Contributor reward & institutional payout system (Deliverables D1–D8)
Plan Duration	75 days · July 21, 2026 → October 3, 2026
Delivery Surfaces	Shared backend service, Android app (Kotlin), iOS app (Swift), Web platform
Network	Stellar testnet (Phases 1–4) → Stellar mainnet (Phases 5–6)

This plan compresses the contracted deliverable roadmap (D1–D8) into a 75-day execution window by running four parallel workstreams. Phases map to the three contract tranches: MVP (Tranche 1), Testnet (Tranche 2) and Mainnet (Tranche 3). External onboarding dependencies (SDP tenancy, Mercuryo partnership/KYC) are initiated in Week 1 to protect the critical path.

1. System Architecture & Workstreams

The product experience lives in the mobile apps, but the settlement system itself is server-side. Treasury multisig signing (HSM/KMS-backed), SDP payout orchestration, Stellar Broker liquidity routing and the deterministic 50/30/20 settlement engine run in one shared backend service. Both mobile apps stay thin and behaviorally identical because they consume the same API. Two surfaces are necessarily web: the Stellar Wallets Kit demo (Freighter is a browser extension) and the Government Settlement Gateway (D8).

Workstream	Stack	Owns
Backend Core	Node.js / TypeScript, @stellar/stellar-sdk, PostgreSQL, Redis	Managed accounts, multisig treasury, delegated signing, SDP, Mercuryo orchestration, Broker routing, settlement engine, SCOUT issuance, indexer, REST/WebSocket API
Android App	Kotlin, Jetpack Compose, Privy Android SDK	Driver onboarding (email/OAuth → wallet), balances, payout history, SCOUT badges, SEP-24 withdrawal webview, notifications
iOS App	Swift, SwiftUI, Privy iOS SDK	Same driver-facing feature set as Android, built natively
Web Platform	TypeScript, Vite/React, Stellar Wallets Kit	Wallet interop demo (Freighter/Lobstr), internal ops console, monitoring surface, Government Settlement Gateway & audit dashboard

2. Phase Timeline Overview

Phase	Days	Dates (2026)	Focus	Deliverables
1. Foundation & Managed Accounts	1–12	Jul 21 – Aug 1	Account layer, Privy onboarding, multisig treasury, app & repo scaffolds	D1
2. Wallet Interop & Payout Rails	13–25	Aug 2 – Aug 14	Stellar Wallets Kit demo, SDP batch payouts, reward screens	D2, D3
3. Off-Ramp & Liquidity Routing	26–40	Aug 15 – Aug 29	Mercuryo SEP-24 withdrawals, Stellar Broker + Aquarius routing	D4, D5
4. Settlement Engine & SCOUT	41–55	Aug 30 – Sep 13	50/30/20 engine, SCOUT reputation assets, settlement indexer	D6
5. Mainnet Readiness & Deployment	56–65	Sep 14 – Sep 23	Security review, HSM prod signing, mainnet rollout, monitoring & alerting	D7
6. Gov Gateway, QA & Handover	66–75	Sep 24 – Oct 3	Government audit dashboard, compliance exports, E2E QA, handover docs	D8

Week-by-Week View

	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11
P1 Foundation											
P2 Interop + SDP											
P3 Off-ramp + Broker											
P4 Settlement + SCOUT											
P5 Mainnet											

P6 Gov Gateway + QA												
---------------------	--	--	--	--	--	--	--	--	--	--	--	--

Overlapping bars reflect parallel workstreams: e.g. while the backend team integrates SDP (P2), the web team builds the Wallets Kit demo and the mobile teams build reward screens. External onboarding (SDP tenancy, Mercury) is requested in Week 1 regardless of phase.

Phase 1: Foundation & Managed Account Infrastructure

Days 1–12 · Jul 21 – Aug 1, 2026 · Covers Deliverable 1

Establish the account layer underpinning the entire settlement system, and stand up all four codebases with CI/CD so every later phase ships into working scaffolds. Kick off SDP and Mercury onboarding paperwork immediately (needed in Phases 2–3).

Workstream	Scope of Work
Backend Core	- Repository, environments and CI/CD for all four codebases; testnet configuration - Deploy protocol-governed distribution accounts: Partner Revenue, Driver Pool, PathPulse Treasury - Multi-signature treasury configuration (threshold $\geq 2/3$ signatories) on testnet - Privy server-side integration: email/OAuth → managed Stellar wallet provisioning - Delegated transaction construction & signing API; KMS-backed signing environment (dev tier)
Android (Kotlin)	- Jetpack Compose app scaffold, design system, networking layer - Privy Android SDK onboarding flow: email/OAuth sign-up → embedded Stellar wallet - Encrypted local storage and session management
iOS (Swift)	- SwiftUI app scaffold mirroring Android information architecture - Privy iOS SDK onboarding flow; Keychain-backed session storage
Web Platform	- Monorepo baseline (extend existing d2-wallet-interop Vite/TS setup); shared UI kit - Internal ops shell with authentication (used by all later phases)

Exit Criteria (measurable)

- Multisig treasury accounts live on testnet with threshold config validated
- Email → Stellar wallet flow functional inside both mobile apps via Privy
- Delegated signing flow validated end-to-end; transactions publicly verifiable on Horizon
- CI pipelines green on all four repos

Note: Treasury/HSM scope carries a human gate — key material and signer set are approved manually, never auto-provisioned.

Phase 2: Wallet Interoperability & Payout Rails

Days 13–25 · Aug 2 – Aug 14, 2026 · Covers Deliverables 2 & 3

Open the system to the wider Stellar ecosystem (external wallets for operators and partners) and stand up the high-volume payout engine. Web and backend run in parallel; mobile teams build the driver-facing reward surfaces on top of Phase 1 accounts.

Workstream	Scope of Work
Backend Core	• Stellar Disbursement Platform (SDP) deployment and payout API integration • Batch payout orchestration for contributor reward distribution • Payout reconciliation, monitoring hooks, error handling and retry logic • Wallet session management APIs for external wallet connections
Web Platform	• Stellar Wallets Kit connector integration • External wallet authentication flows: Freightier (extension) and Lobstr • Reviewer-facing demo: connect wallet → sign & execute testnet transaction
Android (Kotlin)	• Reward balance and payout history screens (Horizon/API-backed) • Push notifications for received payouts
iOS (Swift)	• Reward balance and payout history screens; payout push notifications

Exit Criteria (measurable)

- External wallets connect via Stellar Wallets Kit and sign testnet transactions
- Functional demo accessible to reviewers
- SDP testnet payout batches executed; stablecoin rewards land in contributor accounts
- Payout reconciliation logs verified; end-to-end payout flow demonstrated

Note: SDP tenant onboarding is an external dependency — requested in Week 1; integration work proceeds against the sandbox meanwhile.

Phase 3: Fiat Off-Ramp & Liquidity Routing

Days 26–40 · Aug 15 – Aug 29, 2026 · Covers Deliverables 4 & 5

Give drivers a regulated path from stablecoin rewards to local currency, and give the treasury a path from local-currency funding to USDC. Mercuryio owns KYC, compliance and fiat custody, keeping the compliance boundary outside PathPulse.

Workstream	Scope of Work
Backend Core	• Mercuryio SEP-24 API integration (sandbox): interactive withdrawal flows • Fiat conversion orchestration; off-ramp event indexing linked to settlement batch IDs • Stellar Broker routing layer integration; connection to Aquarius AMM pools • Path payment routing for multi-currency conversion (local currencies → USDC) • Treasury routing logic for incoming non-USDC government funds
Android (Kotlin)	• Withdraw flow: SEP-24 hosted webview (KYC + conversion), status tracking, receipts
iOS (Swift)	• Withdraw flow: SEP-24 hosted webview, status tracking, receipts
Web Platform	• Ops console: payout batch views, off-ramp reconciliation dashboard

Exit Criteria (measurable)

- Testnet withdrawal flow completes via Mercuryio sandbox from both mobile apps
- Stablecoin → fiat conversion simulation validated; off-ramp events linked to settlement transactions
- Asset conversions executed via Stellar Broker on testnet; path payments verified on Horizon

Note: Mercuryio partnership/KYC onboarding is an external dependency — initiated in Week 1 and tracked as a critical-path risk.

Phase 4: Deterministic Settlement Engine & SCOUT Reputation

Days 41–55 · Aug 30 – Sep 13, 2026 · Covers Deliverable 6

Implement the revenue distribution core: deterministic 50% Authorities / 30% Driver Rewards / 20% Treasury splits using native Stellar multi-operation transactions, plus the SCOUT on-chain reputation layer. The settlement indexer built here becomes the data source for the Phase 6 government dashboard.

Workstream	Scope of Work
Backend Core	- Multi-operation settlement transaction construction enforcing 50/30/20 splits - Incentive pool routing with reputation-based multipliers (1.0x / 1.2x / 1.5x) - SCOUT1/2/3 Classic Asset issuance (AUTH_REQUIRED + AUTH_REVOCABLE + AUTH_CLAWBACK_ENABLED) - Tier assignment and revocation driven by PulseGen validation scores - Batch settlement orchestration; settlement indexer v1 (feeds D8)
Android (Kotlin)	SCOUT tier badge UI, multiplier display, earnings breakdown per settlement batch
iOS (Swift)	SCOUT tier badge UI, multiplier display, earnings breakdown
Web Platform	Internal settlement explorer: batch drill-down Treasury → Split → SDP → Driver (gov dashboard foundation)

Exit Criteria (measurable)

- Revenue split transactions execute on testnet with correct 50/30/20 distribution
- Test drivers hold SCOUT tiers visible in Stellar wallets on testnet
- Multipliers applied correctly in settlement batches; settlement validated end-to-end

Phase 5: Mainnet Readiness & Production Deployment

Days 56–65 · Sep 14 – Sep 23, 2026 · Covers Deliverable 7

Transition the fully-operational testnet system to Stellar mainnet with production-grade key management, monitoring and staged rollout. Mobile apps are hardened and submitted for store review in this window to absorb review lead time.

Workstream	Scope of Work
Backend / Infra	• Security review and treasury key ceremony (human-gated); HSM/KMS production signing environment • Mainnet deployment: managed accounts, multisig treasury, SDP production payouts • Mercury production off-ramp; Stellar Broker routing in production • SCOUT Classic Assets issued on mainnet • Monitoring stack: metrics, dashboards, alerting, on-call runbooks; staged low-value pilot payouts
Android (Kotlin)	• Release hardening, crash reporting, mainnet configuration, Play Store submission
iOS (Swift)	• Release hardening, crash reporting, mainnet configuration, App Store submission
Web Platform	• Monitoring surface and internal status page wired to alerting stack

Exit Criteria (measurable)

- Mainnet reward payouts executed successfully (pilot batch)
- Off-ramp transactions processed through Mercury production infrastructure
- Cross-asset settlements executed on mainnet via Stellar Broker
- SCOUT assets issued on mainnet and visible in Stellar wallets
- Monitoring and alerting stack operational

Note: Mainnet deployment is human-gated: no mainnet transaction is constructed or signed without explicit sign-off.

Phase 6: Government Settlement Gateway, QA & Handover

Days 66–75 · Sep 24 – Oct 3, 2026 · Covers Deliverable 8 + E2E QA

Ship the institutional transparency layer used by government and ecosystem partners, run full-system QA, and hand over documentation. The dashboard reads from the settlement indexer hardened since Phase 4, so this phase is assembly and polish, not greenfield build.

Workstream	Scope of Work
Web Platform	- Government settlement dashboard: revenue distribution tracking with end-to-end traceability (Treasury → Settlement Split → SDP → Driver) - On-chain transaction audit trail views; partner access controls - Compliance reporting exports (CSV / PDF)
Backend Core	Settlement indexing hardened for production query load; export APIs
QA (all surfaces)	- End-to-end regression: onboarding → reward payout → settlement split → withdrawal, on both apps - Load tests on batch payouts and indexer; security pass on external surfaces
Handover	Architecture documentation, runbooks, API reference, credential handover, partner walkthrough session

Exit Criteria (measurable)

- Dashboard live in production; partner integration demonstrated
- Revenue splits traceable from treasury deposit to individual driver payout
- Settlement indexing operational; compliance export reports generated
- E2E QA suite passing across Android, iOS, web and backend; handover documentation delivered

7. External Dependencies & Risks

Risk / Dependency	Impact	Mitigation
SDP tenant onboarding (external)	Blocks Phase 2 payout rails	Request in Week 1; build against SDP sandbox in parallel; keep integration behind a payout-provider interface
Mercuryo partnership & KYC approval (external)	Blocks Phase 3 off-ramp go-live	Initiate commercial/compliance onboarding in Week 1; complete full sandbox integration regardless
App Store / Play Store review lead time	Could delay Phase 6 public availability	Submit hardened builds during Phase 5; use staged rollout / TestFlight for pilot users
Treasury key ceremony scheduling	Blocks mainnet cutover (Phase 5)	Human-gated by design; schedule signatories in advance during Phase 4
Aquarius liquidity depth in target corridors	Poor conversion rates for some local currencies	Validate corridor liquidity on testnet analogs in Phase 3; fall back to staged treasury conversions
PulseGen score feed availability	SCOUT tier assignment (Phase 4) needs real scores	Define score-feed contract in Phase 1; use synthetic scores until the live feed lands

8. Working Model & Assumptions

- Team shape: 1 backend/Stellar lead, 1 Android engineer, 1 iOS engineer, 1 full-stack web engineer, shared QA/DevOps.
- 75 calendar days including weekends as buffer; phase dates are targets, exit criteria are the gates.
- All Phase 1–4 work runs exclusively on Stellar testnet; mainnet artifacts only appear in Phases 5–6 behind human gates.
- Progress is tracked in Linear (CUB-5 → CUB-12); each phase's exit criteria map 1:1 to issue acceptance checklists.
- The contract's measure-of-completion items are treated as the authoritative definition of done for each deliverable.